St. Anna Kinderkrebsforschung
CHILDREN'S CANCER RESEARCH INSTITUTE

Modern Dependency & Environment Managers

JAN OPPELT

Computational Biology Meeting

Sep 01, 2026

www.ccri.at

Conda and venv already work – so why bother?

- They do work. But well-known pain points motivated a faster, lock-first generation of tools.

Speed is solved — base env isn't

- Conda ≥ 23.10 defaults to the fast libmamba solver. But it still centers a mutable base environment that's easy to pollute with project deps.

conda + pip friction

- Mixing conda install and pip install in one environment can create hard-to-resolve dependency conflicts.

No lockfile by default

- pip + venv don't maintain a project lock. requirements.txt records intent; pip freeze snapshots a machine, not a resolved graph.

Reproducibility is manual

- Traditional workflows don't keep a fully-resolved, version-controlled environment in sync on every dependency change.

The fix

- **uv** and **pixi** are written in Rust and built around fast parallel resolution plus an automatically maintained lockfile that records the resolved dependency graph – not just what you asked for.

Disclaimer

- I was briefly introduced to uv and pixi just a few weeks back. The main purpose of this talk isn't to convince you about switching or give you all the details, but to raise awareness that there are other tools beyond conda or venv.

- Consider this more like as an overview of methods you might not know about (I didn't until recently).

Manifest vs. lockfile: the one idea to remember

Manifest (What you requested)

- Human-edited, short, often version ranges
- Records intent
- pyproject.toml · pixi.toml · environment.yml

Lockfile (The resolved result)

- Machine-written: exact + transitive deps, hashes
- Reproduces the environment
- uv.lock · pixi.lock · renv.lock · pylock.toml

Reproducibility comes from the lockfile, not the manifest

- Two people installing “the same” environment.yml can still get different versions. A lockfile is necessary – but not sufficient: interpreter, target platform, system libraries, GPU stack, and the OS/container layer still matter.

Two jobs – and tools that often do both

Project / environment managers

- Build a reproducible environment for one project.
- uv · pixi · renv

Application managers

- Install CLI tools in isolated envs, expose on PATH.
- uv tool/pipx · pixi global/condax/conda-global

Note: Not mutually exclusive – uv (uv + uv tool) and pixi (pixi + pixi global) each cover both roles.

Overview

Tool	Role	Ecosystem	Lang	Lockfile
`uv`	Project / env	PyPI / Python	Rust	uv.lock
`uv tool / uvx`	Application	PyPI / Python	Rust	no project lock
`pixi`	Project / env	Conda + PyPI	Rust	pixi.lock
`pixi global`	Application	Conda	Rust	manifest only
`renv`	Project / env	CRAN / Bioconductor / GitHub	R	renv.lock
`pipx`	Application	PyPI / Python	Python	manifest only*

*pipx (and pixi global / conda-global) also have explicit manifests / PEP 751 workflows – but a manifest is not a project lockfile.

uv – fast, all-in-one Python

One Rust tool for Python versions, virtual envs, dependencies, and CLI tools

Pros

- Extremely fast; replaces pip, virtualenv, pyenv, pip-tools, poetry.
- Maintains uv.lock automatically – a universal lock across platforms & Python versions.
- uv run executes inside the managed env – no activation step to remember.
- Reads pyproject.toml; imports an existing requirements.txt.

Cons

- Python / PyPI only – cannot install samtools, bwa, bedtools, or R.
- Source-only packages build from source: need a compiler and -dev headers.
- Heavy native / GPU stacks need the right extra index configured.
- Still 0.x – minor releases may break; pin the uv version in CI.

pixi – the bioinformatics default

Conda-ecosystem, project-first: conda-forge + bioconda, with a lockfile every time

Pros

- Installs compiled tools, system libs, Python, R & mixed languages from conda-forge / bioconda.
- Maintains pixi.lock automatically – and needs no mutable base environment.
- PyPI deps resolved with uv's resolver, coordinated with the Conda side, in one lock.
- One lockfile can hold multiple declared platforms; imports environment.yml.

Cons

- Introduces a pixi project model / manifest; the project is still evolving.
- Multi-platform locking can't invent a build that doesn't exist (e.g. osx-arm64 bioconda).
- Large multi-platform pixi.lock can cause git merge conflicts – never hand-merge it.
- Conda↔PyPI name mapping can occasionally pick the wrong package.

renv – reproducible R libraries

Per-project R library + renv.lock. The whole workflow is init → snapshot → restore

Pros

- Each project gets its own private library; records exact versions & sources in renv.lock.
- Installs from CRAN, Bioconductor, GitHub and more – all pinned in one lock.
- Discovers package dependencies automatically from your code.
- Configurable for fast binary installs (e.g. Posit Public Package Manager).

Cons

- R packages only – does NOT install or manage R itself, or system libraries.
- Compiled packages (sf, terra, rJava, xml2…) need external system libs.
- Bioconductor is coupled to a specific R / Bioconductor release.
- Restoring old, source-only versions can be slow and needs a toolchain.

Isolated CLI tools – one env per app, on your PATH 1/2

Each application gets its own environment.

uv tool / uvx (Python)

- pipx built into uv. Fast; uvx runs a tool in a throwaway env without installing it.

pipx (Python)

- The classic. One venv per app; pipx run for one-offs; pipx inject for plugins.

pixi global (Conda)

- conda/pipx-style for the Conda ecosystem – compiled + non-Python tools. pixi-global.toml.

conda-global (Conda)

- Newer conda-incubator plugin. Native trampoline launchers; ~/.conda/global.toml.

condax (Conda)

- The older "pipx for conda." Lightly maintained → prefer pixi global / conda-global.

Isolated CLI tools – one env per app, on your PATH 2/2

Key caveat

- A global-tool manifest is not a project lockfile. Don't rely on it for research-critical tools.
- Isolation ≠ reproducibility. If a result depends on a tool, make it a project dependency(uv/pixi) and run it with uv run / pixi run.

Which one should I use?

When reproducibility and portability are the priority

Pure-Python project *package, script, or analysis with only PyPI deps*

➔ uv

Compiled tools, system libs, mixed languages, Bioconda *the common bioinformatics case*

➔ pixi

R-only analysis *reproducing the R package library*

➔ renv

CLI tool that is part of the analysis *affects results / is in the pipeline*

➔ project dep – don't install globally

personal utility, not part of results *tools you want available everywhere*

➔ uv tool / pixi global

You don't have to pick just one

The golden rule

- If the result of the research depends on a program, make that program a project dependency and put it in the project lockfile.
  - Lock it. Test that it restores from a clean environment.
- A reproducibility-focused setup mixes tools by ecosystem – each project's software lives in its own lockfile, and personal tooling stays out of the way.

Containers: capture the system, not just the packages

A lockfile pins packages – but NOT the OS, system libraries, compilers, kernel, or GPU driver. For papers, pipelines and regulated work, wrap the environment in a container.

The workflow:

Manifest + lockfile -> Strict restore -> Image built from pinned inputs -> Preserve the image/digest

Pin the base image by digest

`FROM ubuntu:latest` + unpinned installs = not deterministic.

Complementary layers: lockfile for packages, container for the system. On HPC use Apptainer/Singularity or Podman.

When these managers struggle

Reproduce the whole system

- Reach for a container, or Nix / Guix

HPC: offline nodes, quotas, proxies

- Prime caches on a login node; relocate cache dirs; configure private indexes.

Apple Silicon + bioinformatics

- Expect missing osx-arm64 builds; plan a Linux container.

Compiling from source

- You also need system deps + a compiler these tools don't provide.

Upstream can vanish

- Removed/re-uploaded packages break a restore – keep an image or mirror.

Containers aren't the whole computer

- They share the host kernel & can depend on GPU drivers/hardware.

Key takeaways

- Reproducibility comes from the lockfile, not the manifest.
- uv for Python · pixi for bioinformatics & mixed · renv for R – combine as needed.
- Application-manager isolation ≠ project reproducibility.
- Pin the interpreter, declare your platforms, use strict locked restores in CI.
- Containers are the outermost layer: lockfile for packages, container for the system.
- Golden rule: if the result depends on a program, make it a project dependency and lock it.

For more information...

More information with examples available at:

BiCU-CCRI/20260901_Dependency_managers

Try it yourself:

Code → Codespaces → Create codespace → follow the instructions in exercises/

Additional information

From Maud Plaschka

- Documentation for the ALSF Datalab (single cell pediatric cancer atlas), which is imho really good to start renv and managing the system with Docker/Podman
  Creating Docker images - OpenScPCA Documentation

St. Anna Kinderkrebsforschung
CHILDREN'S CANCER RESEARCH INSTITUTE

St. Anna Kinderkrebsforschung GmbH
St. Anna Children's Cancer Research Institute
Zimmermannplatz 10, 1090 Vienna, Austria
Tel: +43-1-40470
office@ccri.at
www.ccri.at

Science that makes a difference.